Implementation Notes: Automatic Wagtail Presentation Tree from main_app Curator Flows

Project Notes

April 12, 2026

1 Goal and Scope

This implementation introduces an automatic content-structure pipeline between the Django application layer (`main_app`) and Wagtail pages. The objective is to let curators create exhibitions and pieces only through `main_app` forms, while the required Wagtail presentation pages are created programmatically with a deterministic hierarchy.

The solution includes:

- automatic Wagtail subtree creation on exhibition and piece creation,
- explicit linkage between business models and Wagtail page roots,
- disabling manual child-page creation in Wagtail admin,
- draft-by-default page state at creation time.

2 Files Modified and Added

File	Purpose
`main_app/models.py`	Added presentation page models, shared content mixin, and link fields from `Exhibition` and `Piece` to Wagtail roots.
`main_app/views/models_views.py`	Added orchestration methods to create/resolve page trees during `form_valid`.
`main_app/wagtail_hooks.py`	Added hooks to block manual page creation and remove add-child actions from admin UI.
`main_app/templates/main_app/models/wagtail/*.html`	Added templates for newly introduced page types.
`main_app/migrations/0002_exhibitionpage_piecepage_and_more.py`	Schema migration for added models and link fields.
`main_app/scripts/seed_museum.py`	Updated seeding behavior to keep pages unpublished by default.

3 Data Model and Page Model Extensions

3.1 Business model links

In `main_app/models.py`, two link fields were added:

- `Exhibition.presentation_page`: `OneToOneField` to `ExhibitionPage`
- `Piece.presentation_page`: `OneToOneField` to `PiecePage`

These fields make the integration explicit and idempotent: each domain object can be traced to exactly one presentation root page.

3.2 Wagtail hierarchy

The hierarchy is constrained with `parent_page_types` and `subpage_types`:

1. Wagtail root
2. `ExhibitionPage`
3. Children of exhibition: `ExhibitionPublicPage`, `ExhibitionPrivatePage`, `ExhibitionPiecesPage`
4. Under `ExhibitionPiecesPage`: `PiecePage`
5. Children of piece: `PiecePublicPage`, `PiecePrivatePage`

3.3 Shared presentation fields

A reusable abstract mixin `PresentationContentMixin` was introduced with:

- `body` (`RichTextField`)
- `hero_image` (FK to Wagtail image model)

This allows future editorial content expansion without repeating field definitions.

4 Creation Workflow in Views

The orchestration is implemented in `main_app/views/models_views.py`.

4.1 Exhibition creation path

`ExhibitionCreateView.form_valid` now:

1. saves the `Exhibition` object,
2. adds the current user as admin,
3. calls `_ensure_exhibition_presentation_tree()`.

`_ensure_exhibition_presentation_tree()` is responsible for:

- short-circuiting if `presentation_page` already exists,
- creating an `ExhibitionPage` under root,
- creating fixed children with slugs `public`, `private`, `pieces`,
- persisting the link back to `Exhibition.presentation_page`.

4.2 Piece creation path

`PieceCreateView.form_valid` now:

1. binds the new piece to the exhibition from URL context,
2. saves the `Piece`,
3. calls `_ensure_piece_presentation_tree()`.

`_ensure_piece_presentation_tree()`:

- returns early if the piece already has `presentation_page`,
- resolves (or creates) the exhibition `pieces` page,
- creates `PiecePage` and fixed children `public`/`private`,
- stores the link in `Piece.presentation_page`.

5 Helper Methods and Their Rationale

The following helpers were added in `ExhibitionCreateView` and reused by `PieceCreateView`:

- `_unique_child_slug(parent, base)`: generates a non-colliding slug per parent.
- `_get_or_create_child(parent, model_class, title, slug, ...)`: idempotent child creation by slug.
- `_save_as_draft(page)`: stores a revision and guarantees not-live status.

These methods reduce duplication, centralize invariants, and prevent accidental duplicate page trees under retries.

6 Unpublished by Default (Draft State)

6.1 Problem encountered

In this project configuration, newly inserted pages could still appear live after insertion/revision handling.

6.2 Final enforcement strategy

A dedicated method `_save_as_draft(page)` was introduced:

1. call `save_revision()`,
2. if `page.live` is true, call `unpublish()`.

This explicit post-save unpublish guarantees draft state regardless of local Wagtail behavior.

The same draft policy was also applied in `main_app/scripts/seed_museum.py` to keep seeded pages unpublished.

7 Wagtail Admin Restrictions

In `main_app/wagtail_hooks.py`, hooks were added to enforce workflow discipline:

- block manual page creation requests in admin,
- remove add-child actions from page listings/action menus.

This ensures curators create structure only through `main_app` and prevents ad-hoc tree drift.

8 Templates Added

Minimal render templates were added for all introduced page types under: `main_app/templates/main_app/mod`

Current templates are intentionally simple, serving as placeholders for future editorial customization.

9 Behavioral Guarantees

After implementation, the system provides:

- deterministic hierarchy for each exhibition and piece,
- idempotent creation (re-entry safe),
- default draft state for generated pages,
- no manual child creation path in Wagtail admin.

10 Known Constraints

- A `Piece` is currently modeled as belonging to one exhibition context in this workflow.
- Reusing the same piece across multiple exhibitions would require model redesign (e.g., many-to-many plus revised presentation linking strategy).

11 Validation Summary

Validation was done via:

- migration generation and apply,
- `python manage.py check`,
- smoke tests through create views verifying expected subtree and `live=False` across generated nodes.

12 Possible Next Iterations

- add a formal management command for deterministic backfill of legacy exhibitions/pieces,
- switch from simple rich text fields to structured content blocks if presentation complexity grows,
- add tests that assert full tree structure and publication state per create operation.